

tf.compat.v1.batch_to_space_nd Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/batch_to_space_nd

BatchToSpace for N-D tensors of type T.

Function Signatures

```
tf.compat.v1.batch_to_space_nd( input: Annotated[Any,
TV_BatchToSpaceND_T], block_shape: Annotated[Any,
TV_BatchToSpaceND_Tblock_shape], crops: Annotated[Any,
TV_BatchToSpaceND_Tcrops], name=None ) -> Annotated[Any,
TV_BatchToSpaceND_T]
```

Code Examples

```
tf.compat.v1.batch_to_space_nd(  
    input: Annotated[Any, TV_BatchToSpaceND_T],  
    block_shape: Annotated[Any, TV_BatchToSpaceND_Tblock_shape],  
    crops: Annotated[Any, TV_BatchToSpaceND_Tcrops],  
    name=None  
) -> Annotated[Any, TV_BatchToSpaceND_T]
```

```
tf.compat.v1.batch_to_space_nd(  
    input: Annotated[Any, TV_BatchToSpaceND_T],  
    block_shape: Annotated[Any, TV_BatchToSpaceND_Tblock_shape],  
    crops: Annotated[Any, TV_BatchToSpaceND_Tcrops],  
    name=None  
) -> Annotated[Any, TV_BatchToSpaceND_T]
```

```
[[[1]], [[2]], [[3]], [[4]]]
```

```
[[[1]], [[2]], [[3]], [[4]]]
```

```
x = [[[1], [2]], [[3], [4]]]
```

```
x = [[[1], [2]], [[3], [4]]]
```

```
[[[1, 2, 3]], [[4, 5, 6]], [[7, 8, 9]], [[10, 11, 12]]]
```

```
[[[1, 2, 3]], [[4, 5, 6]], [[7, 8, 9]], [[10, 11, 12]]]
```

Documentation

BatchToSpace for N-D tensors of type T.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.manip.batch_to_space_nd`

This operation reshapes the "batch" dimension 0 into $M + 1$ dimensions of shape `block_shape + [batch]`, interleaves these blocks back into the grid defined by the spatial dimensions `[1, ..., M]`, to obtain a result with the same rank as the input. The spatial dimensions of this intermediate result are then optionally cropped according to `crops` to produce the output. This is the reverse of `SpaceToBatch`. See below for a precise description.

This operation is equivalent to the following steps:

- Reshape input to reshaped of shape:

```
[block_shape[0], ..., block_shape[M-1],  
batch / prod(block_shape),  
input_shape[1], ..., input_shape[N-1]]
```

- Permute dimensions of reshaped to produce permuted of shape

```
[batch / prod(block_shape),
```

input_shape[1], block_shape[0],

...,

input_shape[M], block_shape[M-1],

input_shape[M+1], ..., input_shape[N-1]]

- Reshape permuted to produce reshaped_permuted of shape

[batch / prod(block_shape),

input_shape[1] * block_shape[0],

...,

input_shape[M] * block_shape[M-1],

input_shape[M+1],

...,

input_shape[N-1]]

- Crop the start and end of dimensions [1, ..., M] of

reshaped_permuted according to crops to produce the output of shape:

[batch / prod(block_shape),

input_shape[1] * block_shape[0] - crops[0,0] - crops[0,1],

...,

input_shape[M] * block_shape[M-1] - crops[M-1,0] - crops[M-1,1],

input_shape[M+1], ..., input_shape[N-1]]

Reshape input to reshaped of shape:

[block_shape[0], ..., block_shape[M-1],

batch / prod(block_shape),
input_shape[1], ..., input_shape[N-1]]

Permute dimensions of reshaped to produce permuted of shape
[batch / prod(block_shape),

input_shape[1], block_shape[0],
...,
input_shape[M], block_shape[M-1],
input_shape[M+1], ..., input_shape[N-1]]

Reshape permuted to produce reshaped_permuted of shape
[batch / prod(block_shape),

input_shape[1] * block_shape[0],
...,
input_shape[M] * block_shape[M-1],

input_shape[M+1],
...,
input_shape[N-1]]

Crop the start and end of dimensions [1, ..., M] of
reshaped_permuted according to crops to produce the output of shape:
[batch / prod(block_shape),

input_shape[1] * block_shape[0] - crops[0,0] - crops[0,1],
...,
input_shape[M] * block_shape[M-1] - crops[M-1,0] - crops[M-1,1],

`input_shape[M+1], ..., input_shape[N-1]]`

Some examples:

(1) For the following input of shape `[4, 1, 1, 1]`, `block_shape = [2, 2]`, and `crops = [[0, 0], [0, 0]]`:

The output tensor has shape `[1, 2, 2, 1]` and value:

(2) For the following input of shape `[4, 1, 1, 3]`, `block_shape = [2, 2]`, and `crops = [[0, 0], [0, 0]]`:

The output tensor has shape `[1, 2, 2, 3]` and value:

(3) For the following input of shape `[4, 2, 2, 1]`, `block_shape = [2, 2]`, and `crops = [[0, 0], [0, 0]]`:

The output tensor has shape `[1, 4, 4, 1]` and value:

(4) For the following input of shape `[8, 1, 3, 1]`, `block_shape = [2, 2]`, and `crops = [[0, 0], [2, 0]]`:

The output tensor has shape `[2, 2, 4, 1]` and value:

Returns

